



Digital Image Processing

Prof. Zohreh Azimifar

Homework 4

Topics: Morphology & Shape Analysis

Submission Format: PDF Report + Source Code (.zip)

June 2026

General Rules & Constraints

- **The "From Scratch" Rule:** All core morphological algorithms must be implemented from scratch using fundamental mathematical and array operations.
 - **FORBIDDEN:** Built-in computer vision functions for morphology (e.g., `cv2.erode`, `cv2.dilate`, `cv2.morphologyEx`, `scipy.ndimage.binary_erosion`, `skimage.morphology.skeletonize`). Using these for the core pipeline will result in an automatic **score of ZERO** for that section.
 - **ALLOWED:** `cv2.imread`, `cv2.imwrite`, `cv2.cvtColor`, `scipy.ndimage.label` (only for counting connected components after morphological processing), numpy array operations, and matplotlib for plotting.
- **Vectorization & Performance:** Python nested for loops over image pixels are extremely slow. You are strictly required to use NumPy vectorization (e.g., array slicing, boolean masking, padding) to optimize your execution time.
- **Execution Profiling:** You must calculate and print the execution time for each major part of your algorithm.
- **Dataset:** Use `input_images/blobs/blobs.png` for Section 1, and the provided `input_images/PCB/` directory for Section 2.

Part 1: Fundamental Binary Morphology & Topological Analysis

Input: input_images/blobs/blobs.png

In this section, you will implement the foundational building blocks of Mathematical Morphology as taught in the lectures and apply them to perform topological analysis on a binary image of arbitrary shapes.

1.1 Basic Operations & Structuring Elements (SE)

Implement the fundamental morphological operations from scratch. Your functions must accept dynamic SE shapes and sizes.

1. **SE Generator:** Create a function to generate flat structuring elements (e.g., 3×3 square, 5×5 cross, radius-3 disk).
2. **Erosion & Dilation:** Implement $A \ominus B$ and $A \oplus B$.
3. **Opening & Closing:** Implement $A \circ B$ and $A \cdot B$ using your erosion and dilation functions.
4. **Evaluation:** Apply these operations to blobs.png. In your report, compare the effects of using a 3×3 square SE versus a 5×5 disk SE on object boundaries, noise removal, and gap filling.

1.2 Region Filling (Hole Filling)

Implement the morphological region filling algorithm from scratch as defined in the course slides:

$$X_k = (X_{k-1} \oplus B) \cap A^c$$

- Generate a binary mask of *only the holes* present inside the shapes in blobs.png.
- Generate the final image where all holes within the blobs have been completely filled.

1.3 Topological Categorization

Using your from-scratch morphological tools (combined with logical operators like XOR, AND, OR), design a pipeline to categorize the objects in blobs.png based on their topology.

1. **Count Total Shapes:** Report the total number of connected foreground objects. *(You may use `scipy.ndimage.label` on your final morphological masks to separate components and find bounding box coordinates).*
2. **Shapes WITH Holes:** Morphologically extract and isolate only the objects that contain at least one hole. Plot the original image with **Green bounding boxes** drawn around these specific objects and report the count.
3. **Shapes WITHOUT Holes:** Morphologically extract and isolate only the solid objects (zero holes). Plot the original image with **Blue bounding boxes** drawn around these specific objects and report the count.

Part 2: Industrial Application - PCB Defect Detection

Input Directory: input_images/PCB/ (Contains reference image and various defect images like 01_short_01.jpg, 01_open_circuit_01.jpg, etc.)

Printed Circuit Board (PCB) manufacturing relies heavily on automated optical inspection. You will build a morphology-based visual inspection system to detect and localize structural differences between a clean reference PCB and defective test PCBs.

2.1 Preprocessing & Binarization

1. Load the reference image and the test image.
2. Implement a binarization method from scratch (e.g., Global thresholding, Otsu's method, or Adaptive Thresholding) to extract the copper tracks.

2.2 Morphological Registration & Cleaning

Raw difference images (XOR) will contain massive amounts of noise due to slight pixel misalignments, illumination variations, and aliasing.

1. Compute the binary difference (e.g., using bitwise XOR) between the reference mask and the test mask.
2. Use **Opening** and **Closing** with carefully chosen structuring elements to eliminate false positives (registration noise) while preserving true defects.
3. **Top-Hat Transform:** Implement White Top-Hat ($A - (A \circ B)$) and Black Top-Hat ($((A \cdot B) - A)$) from scratch to highlight exceptionally bright or dark localized anomalies (like spurious copper or missing holes) before XORing.

2.3 Defect Localization

Process the 5 provided defect images (missing_hole, open_circuit, short, spurious_copper, spur).

1. For each image, isolate the defect using your morphological pipeline.
2. Extract the bounding box coordinates (xmin, ymin, xmax, ymax) of the remaining defect components.

2.4 Quantitative Evaluation (Using XML Ground Truth)

The provided .xml files contain the Ground Truth (GT) bounding boxes for the defects. You must evaluate the accuracy of your morphology-based object detector.

1. **XML Parsing:** Write a brief script (e.g., using Python's built-in xml.etree.ElementTree) to parse the corresponding .xml file for each test image and extract the GT bounding box coordinates.

2. **IoU Calculation:** Implement a function to calculate the **Intersection over Union (IoU)** between your algorithm's predicted bounding box and the GT bounding box.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

3. **Evaluation Metric:** For each of the 5 images, print the computed IoU. Define a detection as a **True Positive (TP)** if the $\text{IoU} > 0.5$. Discuss any False Positives (noise) or False Negatives (missed defects).
4. **Output:** For each image, plot the original defective color image containing:
 - A **Green bounding box** representing the Ground Truth (from XML).
 - A **Red bounding box** representing your Prediction.
5. **Discussion:** In your report, explain how the size of your Structuring Element dictates the algorithm's sensitivity (i.e., avoiding false alarms vs. missing small defects like spurs), referencing your IoU scores.

Part 3: Structural Analysis - Skeletonization & Pruning

3.1 Pattern Matching: Isolated Pairs (Hit-or-Miss Warm-up)

Input: An arbitrary 2D binary array (matrix).

1	1	1	0	1
0	1	0	1	0
0	1	1	1	1
1	1	1	1	0
0	0	1	1	1

Before implementing iterative skeletonization, you will use the **Hit-or-Miss Transform** to detect specific local patterns in a matrix.

Write a program that takes an arbitrary 2D matrix (consisting of 0s and 1s) and finds the exact indices of all **isolated pairs of connected zeros**.

- **Definition:** An "isolated pair of connected zeros" refers to exactly two 0 elements that are connected to each other (horizontally, vertically, or diagonally), with absolutely **no other 0 elements** connected to either of them in their 8-way neighborhood.

Tasks & Constraints:

1. **Morphological Approach:** You must solve this problem using your from-scratch **Hit-or-Miss** transform ($A \circledast B$). Do not use standard iterative for loops with if/else conditions to check neighborhoods.
2. **Structuring Elements (SE):** Design the specific pair of Structuring Elements (B_1 for the foreground, B_2 for the background) required to detect these exact isolated pairs. Keep in mind that pairs can be oriented horizontally, vertically, or diagonally (you may need to design multiple SEs and combine the results).
3. *(Note: Standard morphology usually treats 1 as foreground. You may choose to invert the input matrix first so that 0s become 1s).*
4. **Output:** Return a column matrix (or a structured list) containing the (row, column) indices of the detected pairs. In your PDF report, display a small toy matrix (like the one in the assignment description) and print your algorithm's output to show that it works.

Input: Binarized PCB Reference Image

To analyze the structural integrity of PCB tracks (e.g., to ensure tracks meet minimum width requirements or to map circuit topology), we extract their medial axis (skeleton).

3.2 Skeletonization (Thinning)

Implement morphological Thinning to find the skeleton of the PCB tracks. As taught in the lectures, thinning can be achieved using the Hit-or-Miss transform $(A \circledast B)$:

$$A \otimes B = A - (A \circledast B)$$

1. Implement the Hit-or-Miss operator from scratch.
2. Define the sequence of structuring elements (e.g., the 8 Golay alphabet masks for thinning).
3. Iteratively thin the binary PCB tracks until convergence ($X_k = X_{k-1}$).

3.3 Pruning

Skeletonization often leaves undesirable "parasitic spurs" due to boundary irregularities.

1. Implement morphological Pruning from scratch.
2. Use the endpoint-detection structuring elements to thin the skeleton for a specific number of iterations (e.g., 8 iterations).
3. **Output:** Display a side-by-side comparison of:
 - The cleaned binary PCB tracks.
 - The raw skeleton.
 - The pruned skeleton.

Report Structure & Deliverables - Deadline: 10 July 2026

Submit a single .zip file containing your source code (Python scripts or Jupyter Notebook) and an academic PDF report.

Naming convention: HW4_Student1_Student2_Student3.zip

Your PDF Report Must Include:

- **Chapter 1: Problem Definition**
 - Brief introduction to the assignment and the role of morphology in industrial and topological analysis.
- **Chapter 2: Methodology**
 - Mathematical formulations of your implemented morphological operations (Set theory definitions).
 - Explanation of the vectorization techniques used to eliminate slow loops in Erosion/Dilation.
 - Explanation of the logic used for topological categorization (Section 1.3).
- **Chapter 3: Results and Discussion**
 - Visual outputs for each task (Before/After morphology, defect bounding boxes, skeletons).
 - Provide a table detailing the execution time (in seconds) for your from-scratch Erosion, Dilation, and Skeletonization functions.
 - Discuss the challenges of detecting small defects (like spurs) without triggering false positives on noisy edges.
- **Chapter 4: Conclusion**
 - Summary of the main findings and challenges faced during from-scratch implementations.